# UNIDAD 1 - SEMANA 4
# RESUMEN DE CLASE

**Nombre:** Doménica Rojas
**Ciclo:** Quinto “A”
**Asignatura:** Sistemas Digitales

## Tema: Interrupciones externas en el ESP32

### 1. Del sondeo a la interrupción

En el **sondeo (polling)**, el programa comprueba repetidamente un pin mediante digitalRead() dentro de loop(). Si el ciclo está ocupado con delay() u otra tarea larga, puede no detectar una pulsación breve.

Con una **interrupción externa**, el periférico GPIO detecta un cambio eléctrico en el pin y avisa a la CPU. Esta pausa temporalmente el código principal, ejecuta una **ISR** y continúa donde quedó. No hace falta que loop() vigile constantemente el botón.

### 2. Flancos y elección de los pines

Un **nivel** indica el estado que permanece (HIGH o LOW); un **flanco** indica el instante del cambio. Con un pulsador conectado a GND y configurado con **`INPUT_PULLUP`**, el estado normal es HIGH y al presionar pasa a LOW.

| Modo | Transición | Con INPUT_PULLUP |
|---|---|---|
| FALLING | HIGH → LOW | Se activa al presionar. |
| RISING | LOW → HIGH | Se activa al soltar. |
| CHANGE | Ambas direcciones | Se activa al presionar y al soltar. |

Las diapositivas recomiendan, entre otros, **GPIO16–19, 21–23, 25–27, 32 y 33**. Los **GPIO34–39** permiten interrupciones, pero son solo entrada y no tienen pull-up interna. Es mejor evitar los pines de *strapping* (0, 2, 5, 12, 15) y los de Flash (6–11).

### 3. Cómo configurar y escribir una ISR

La función **`attachInterrupt()`** se coloca en setup() después de pinMode(). Sus argumentos especifican el pin, la función que debe ejecutarse y el tipo de flanco:

```
pinMode(21, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(21), isrBoton, FALLING);
```

La **ISR** debe ser **muy corta**. Puede activar una bandera o incrementar un contador, pero se deben evitar Serial.println(), delay() y tareas largas; de lo contrario, el sistema puede quedar bloqueado o reiniciarse por el *watchdog*.

### 4. IRAM_ATTR, volatile y patrón bandera

- **`IRAM_ATTR`:** coloca el código de interrupción en IRAM, disponible cuando la caché de Flash no puede utilizarse.

- **`volatile`:** indica que una variable puede cambiar fuera del flujo habitual del programa. Es especialmente importante cuando la ISR la escribe y loop() la lee.
- **Bandera:** una variable volatile bool indica que ocurrió un evento. La ISR la activa y loop() la desactiva antes de procesarlo.
- **Contador:** si importa cuántos eventos ocurrieron, usar una variable contadora en vez de una bandera booleana, porque varios eventos pueden quedar resumidos en un solo aviso.

```
volatile bool alerta = false;

void IRAM_ATTR isrBoton() {
   alerta = true;
}

void loop() {
   if (alerta) {
      alerta = false;
      // atender el evento fuera de la ISR
   }
}
```

## 5. Rebote y esquema completo

Un pulsador mecánico puede generar **varios flancos por una sola pulsación**. La presentación propone un **antirrebote de 50 ms**: dentro de la ISR se consulta millis() y se acepta el nuevo flanco únicamente cuando ha transcurrido suficiente tiempo desde el último válido.

El recorrido del aviso es:

**pulsador → GPIO detecta FALLING → ISR breve en IRAM → bandera volatile → loop() procesa la alerta.**

Esta organización hace que el programa siga siendo responsivo incluso cuando el bucle principal está ocupado.